



UNIVERSITÀ DEGLI STUDI DI SALERNO

Corso di Fondamenti di Intelligenza Artificiale (FIA)

Anno Accademico 2025/26



HAIVE

an AI for the bug-themed strategy game

14 febbraio 2026

Autore	Matricola
Forte Guglielmo	0512119526
Rega Maristella	0512119032
Russo Serena	0512119098
Squitieri Andrea	0512119008

LINK GITHUB: <https://github.com/andrea-sq/hAIve>

Indice

1	Introduzione	2
2	Descrizione del problema	3
2.1	Obiettivi	3
2.2	Specifiche PEAS	3
2.2.1	Caratteristiche dell'ambiente	3
2.3	Analisi del problema	4
2.3.1	Rappresentazione dello stato di gioco	5
2.3.2	Contributi precedenti	5
3	Soluzioni proposte	6
3.1	Esplorazione mosse	6
3.1.1	Potatura alfa-beta	6
3.1.2	Monte Carlo Tree Search	6
3.2	Valutazione stato di gioco	6
3.2.1	Funzioni euristiche	6
3.2.2	NNUE	6
4	Confronto tra le implementazioni	6

1 Introduzione

Hive è un gioco da tavolo strategico per due giocatori, ideato da John Yanni e pubblicato nella prima edizione nel 2001.

Rientra nei giochi deterministici ad informazione perfetta e a somma zero, tuttavia, a differenza di esempi più celebri come il gioco degli scacchi, non possiede una plancia e può essere svolto su qualsiasi superficie piana.

In seguito alla più recente espansione, il numero totale di pezzi è 28, di cui 14 bianchi e 14 neri, raffiguranti un insieme di insetti, aracnidi e isopodi, ognuno con un modo particolare di muoversi. L'obiettivo è catturare l'*ape regina* avversaria, facendo sì che sia completamente circondata da altri pezzi.

Perché Hive? Un'analisi del branching-factor [1] mostra che, negli stadi più avanzati del gioco, il fattore di ramificazione è stabile a 60. Pertanto, la crescita esponenziale di Hive è circa due volte quella degli scacchi, il cui fattore di ramificazione è intorno al 30.

L'interesse nel creare un'intelligenza artificiale in grado di giocare ad Hive non deriva solo dall'elevata complessità, ma anche dalla scarsa notorietà del gioco che lo rende un problema non ancora esplorato esaustivamente.

2 Descrizione del problema

2.1 Obiettivi

Lo scopo del progetto consiste nella realizzazione di un **modello di intelligenza artificiale** in grado di sfidare un altro agente intelligente — artificiale o umano — nel gioco di *Hive*.

2.2 Specifiche PEAS

- **Performance.** Le misure adottate per valutare le prestazioni sono il *tempo* per generare una mossa e le *percentuali di vittoria* contro una generatore di mosse randomiche e altri agenti noti.
- **Environment.** L'ambiente in cui opera l'agente è costituito dall'insieme di tutte le possibili configurazioni che l'alveare può assumere.
- **Actuators.** L'agente agisce sull'ambiente mediante la successiva mossa scelta.
- **Sensors.** L'agente percepisce l'ambiente mediante la sequenza di mosse dell'avversario.

2.2.1 Caratteristiche dell'ambiente

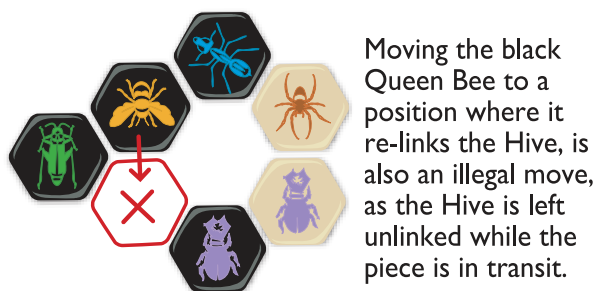
L'ambiente operativo è:

- **Completamente osservabile.** I sensori forniscono accesso allo stato completo dell'ambiente in ogni momento.
- **Deterministico.** Lo stato successivo è completamente determinato da stato corrente e dalla seguente azione scelta.
- **Sequenziale.** Ogni decisione può influenzare tutte quelle successive.
- **Statico.** L'ambiente rimane invariato mentre l'agente delibera.
- **Discreto.** Il numero di percezioni e azioni distinte è finito.
- **Multiagente competitivo** L'ambiente consente la presenza di esattamente due agenti, ognuno dei quali tenta di sconfiggere l'avversario.

2.3 Analisi del problema

Il gioco è costituito da poche e semplici regole, di seguito riassunte per facilitare la comprensibilità del problema.

- Ad ogni turno un giocatore potrà eseguire una delle due possibili mosse:
 1. Far entrare in gioco un insetto (finché ne restano a disposizione);
 2. Muovere un insetto già posizionato (se in grado di farlo).
- Facendo entrare in gioco un insetto, lo si deve posizionare in modo che nessuno dei suoi lati tocchi un insetto del colore avversario già posizionato, ma unicamente insetti del proprio colore. (Fa eccezione la prima mossa di ogni giocatore).
- L'ape regina deve entrare in gioco entro il quarto turno e, in mancanza di essa, i pezzi in gioco non possono muoversi.
- Se un giocatore non può muovere un suo pezzo in campo né aggiungerne di nuovi, passa il turno all'avversario: il gioco continua in questo modo finché il primo giocatore non è nuovamente in grado di fare la sua mossa.
- *One Hive Rule*: i pezzi in gioco devono essere costantemente connessi, non è mai possibile dividere l'alveare in due o più parti.



- *Freedom To Move*: poiché le creature si muovono scorrendo intorno all'alveare, un pezzo può muoversi da o verso una posizione solo se può fisicamente scivolare in quella direzione. Fa eccezione a questa regola il Grillo, ed eventualmente la Zanzara quando lo imita, in quanto può saltare sopra l'alveare. La Coccinella e lo Scarabeo sono in grado di spostarsi da o verso posizioni totalmente circondate muovendosi sopra l'alveare, ma devono comunque rispettare la regola nel caso al secondo livello vi sia un "varco stretto".



2.3.1 Rappresentazione dello stato di gioco

Per modellare lo stato in cui si trova l'alveare sono state adottate due rappresentazioni distinte:

- Una mappa esagonale modulare di raggio 14 (e diametro 29) a cui abbiamo applicato le coordinate assiali per identificare univocamente la posizione di ogni esagono. La scelta del raggio corrisponde al più piccolo numero che permette di posizionare tutti i 28 pezzi e mantenere l'illusione di una mappa infinita, in modo da minimizzare la quantità di spazio necessario a memorizzare ogni stato.
- Un vettore binario che rappresenta la presenza o l'assenza di possibili archi e l'altezza dei pezzi che sono in grado di salire sull'alveare.

2.3.2 Contributi precedenti

Il progetto Mzinga [2] è una collezione di software open-source per agevolare la comunità di sviluppatori e giocatori di Hive.

Offre tra le principali feature l'**Universal Hive Protocol** (UHP) che supporta l'interoperabilità tra software dedicati al gioco. Opera distinguendo tra due principali tipi di programmi: engines e viewers. Un engine ha il compito di comprendere come funziona una partita di Hive. Un viewer semplifica l'interazione dell'utente umano con l'engine.

Abbiamo deciso di implementare **UHP** per semplificare l'interazione e il confronto tra il nostro agente e altri agenti pre-esistenti. Ulteriore beneficio di questa scelta è derivato dall'utilizzo del viewer di Mzinga, che ci ha fornito un'interfaccia di gioco ampiamente utilizzata dalla community.

3 Soluzioni proposte

A partire dall'analisi effettuata, risulta naturale che si tratti di un problema di ricerca con avversario.

Per giungere ad una soluzione è necessario definire due componenti:

- **Algoritmo di esplorazione delle mosse.** Per ricercare la scelta più conveniente, non è sufficiente conoscere le mosse possibili in un dato istante, ma è necessario studiare le implicazioni di queste e quindi le possibili mosse n turni nel futuro.
- **Algoritmo di valutazione dello stato di gioco.** É impensabile esplorare completamente l'albero di gioco a causa dell'elevato *fattore di ramificazione*, che ricordiamo essere pari a 60. Di conseguenza, è necessario poter usufruire di una nozione della bontà di un determinato stato di gioco.

3.1 Esplorazione mosse

3.1.1 Potatura alfa-beta

Negamax

Young Brothers Wait

MTD(f)

3.1.2 Monte Carlo Tree Search

3.2 Valutazione stato di gioco

3.2.1 Funzioni euristiche

3.2.2 NNUE

4 Confronto tra le implementazioni

Riferimenti bibliografici

- [1] Duncan Kampert et al. «Mimicking the Human Approach in the Game of Hive». In: *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. 2021, pp. 1–8. DOI: 10.1109/SSCI50451.2021.9659999.
- [2] Jon Thysell. *Mzinga: open-source software to play the board game Hive*. URL: <https://github.com/jonthysell/Mzinga>.